

M5.1 Comunicação e associação

Associações

As associações representam o equivalente mais próximo dos relacionamentos utilizados no modelo Entidade-Relacionamento, ou seja, seu objetivo é definir a maneira como as classes estão unidas e se relacionam entre si, compartilhando informações – (Guedes, 2006, p. 72)

Um associação representa um relacionamento entre classes, e fornece a semântica comum e a estrutura para muitos tipos de “conexões” entre objetos.

São relacionamentos estruturais entre instâncias e especificam que objetos de uma classe estão ligados a objetos de outras classes.

A associação pode existir entre classes ou entre objetos. Uma associação entre a classe Professor e a classe disciplina (um professor ministra uma disciplina) significa que uma instância de Professor (um professor específico) vai ter uma associação com uma instância de Disciplina. Esta relação significa que as instâncias das classes são conectadas, seja fisicamente ou conceitualmente.

Associações são o mecanismo que permite objetos comunicarem-se entre si. Elas descrevem a conexão entre diferentes classes (a conexão entre os objetos atuais é chamada conexão do objeto, ou link).

Associações podem ter um regra que especifica o propósito da associação e pode ser uni ou bidirecional (indicando se os dois objetos participantes do relacionamento podem mandar mensagens para o outro, ou se apenas um deles sabe sobre o outro).

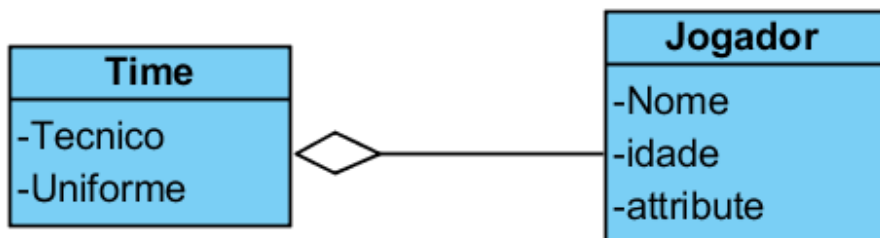
Cada ponta da associação também possui um valor de multiplicidade, que dita como muitos objetos neste lado da associação pode relacionar-se com o outro lado.

Em UML, associações são representadas como linhas conectando as classes participantes do relacionamento, e podem também mostrar a regra e a multiplicidade de cada um dos participantes. A multiplicidade é exibida como um intervalo [min...máx] de valores não negativos, com uma estrela (*) no lado máximo representando infinito.

Agregação e composição

Agregações são um tipo especial de associação no qual as duas classes participantes não possuem em nível igual, mas fazem um relacionamento “todo-parte”. Uma Agregação descreve como a classe que possui a regra do todo, é composta (tem) de outras classes, que possuem a regra das partes. Para Agregações, a classe que age como o todo sempre tem uma multiplicidade de um. Nele, os objetos continuam existir mesmo que o todo seja removido. Assim se temos um time de basquete, e pessoas como jogadores, ambos existirão independentemente

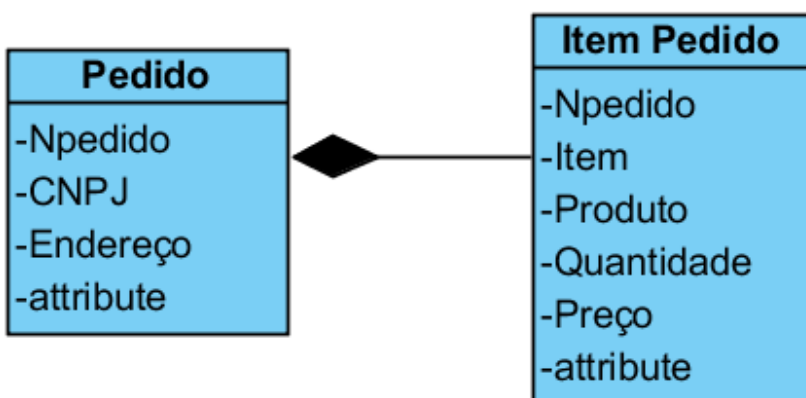
Em UML, Agregações são representadas por uma associação que mostra um romboide no lado do todo.



Composição

Composições são associações que representam agregações muito fortes. Isto significa que Composições formam relacionamentos todo-parte também, mas o relacionamento é tão forte que as partes não pode existir independentes. Elas existem somente dentro do todo, e se o todo é destruído as partes morrem também. A composição tenta representar um vínculo mais forte entre os objetos-todo e os objetos-parte, procurando demonstrar que os objetos-parte têm de pertencer exclusivamente a um único objeto-todo com que se relacionam. Em uma composição, um mesmo objeto-parte não pode associar-se a mais de um objeto-todo. Assim se o todo deixar de existir as partes deixam de existir e se as partes deixarem de existir o todo desaparece. Assim o exemplo mais típico é o do pedido de compras. O Pedido não existirá se não houverem itens, assim como itens apenas não faz sentido se não estiverem agrupados em um pedido.

Em UML, Composições são representadas por um romboide sólido no lado do todo.



Como exemplo de associação podemos criar uma classe chamada motor, onde temos como característica a quantidade de cilindros e o volume do motor em litros. Um a outra classe chamada Carro cujas características são a cor e o motor que o equipa

```

class Motor
{
    public int Cilindros;
    public double Capacidade;
    public Motor(int cil, double cc)
    {
        this.Cilindros = cil;
        this.Capacidade = cc;
    }
}

```

classe motor

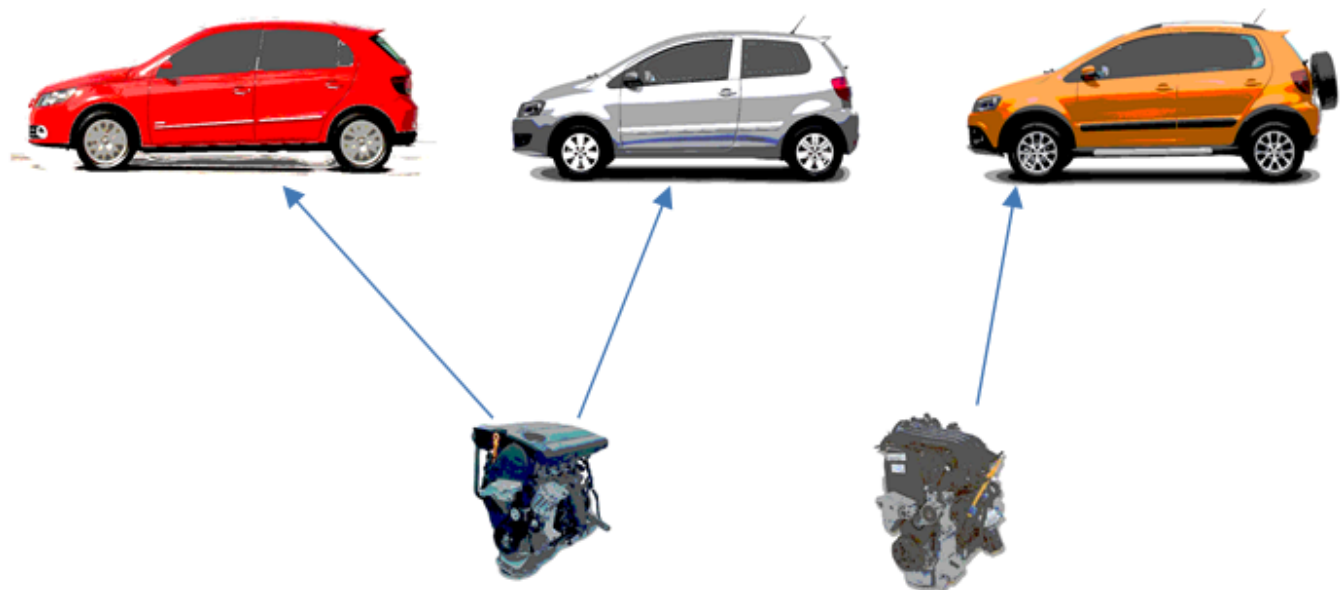
```

class Carro
{
    public string cor;
    public Motor motor;
    public Carro(String cor, Motor motor)
    {
        this.cor = cor;
        this.motor = motor;
    }
}

```

Classe carro

Assim para criarmos três carros, um Gol 1000 vermelho, um Fox 1.0 prata e um CrossFox 1.6 Amarelo



```

class Program
{
    static void Main(string[] args)
    {
        Motor AP1000 = new Motor(4, 1.0);
        Motor AP1600 = new Motor(4, 1.6);
        Carro gol1000 = new Carro("Vermelho", AP1000);
        Carro fox1000 = new Carro("Preto", AP1000);
        Carro crossFox = new Carro("Amarelo", AP1600);
        Console.WriteLine(crossFox.motor.Capacidade);
    }
}

```

Assim o código fica:

Reutilização de classes

Podemos criar novas classes que estendem uma outra classe base se incluirmos uma instância da classe base como um dos campos da nova classe, que será então composta de campos específicos e de uma instância de uma classe base. Para que os métodos da classe base possam ser executados, escreveremos métodos correspondentes na classe nova que chamam os da classe base, dessa forma delegando a execução dos métodos.

Podemos criar novas classes que estendem uma outra classe base se incluirmos uma instância da classe base como um dos campos da nova classe, que será então composta de campos específicos e de uma instância de uma classe base. Para que os métodos da classe base possam ser executados, escreveremos métodos correspondentes na classe nova que chamam os da classe base, desta forma delegando a execução dos métodos. (Santos, 2003, p. 174)

Exemplo: considere a classe *Data*, que representa uma data e a classe *Hora* que representa uma hora. Ambas contêm métodos que iniciam e verificam a validade de seus campos e imprimem os valores de seus campos em um formato apropriado. Com essas classes podemos criar a classe *DataHora* que representa simultaneamente uma data e uma hora, sem que seja necessário reescrever os campos e métodos contidos nas classes *Data* e *Hora*.

Os únicos campos na classe *DataHora* são uma instância da classe *Data* e uma instância da classe *Hora*. Todas as informações que devem ser representadas por uma classe *DataHora* estarão contidas nessas instâncias. Campos adicionais poderiam ser declarados, se fosse necessário.

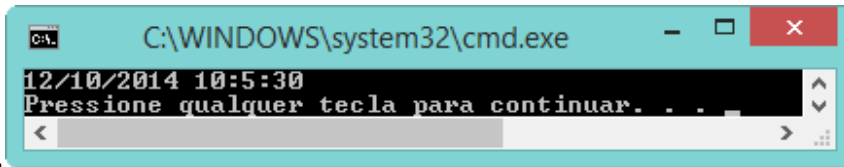
O primeiro construtor da classe *DataHora* recebe seis argumentos, correspondentes ao dia, mês, ano, hora, minuto e segundo que devem ser representados pela classe *DataHora* e repassa esses argumentos para os construtores que iniciarão as instâncias das classes *Data* e *Hora* respectivamente. O construtor da classe *DataHora* delega aos outros construtores a iniciação dos campos. O mesmo acontece com o segundo construtor da classe *DataHora*, exceto que este considera que a hora é, por padrão, meia-noite.

O método *ToString* também delega o seu funcionamento aos métodos *ToString* das classes *Data* e *Hora*, que retornarão *strings* que são concatenadas para criar o resultado da chamada do método *ToString* da classe *DataHora*.

Implementando mecanismo de delegação.

```
class Program
{
    static void Main(string[] args)
    {
        DataHora agora = new DataHora(10, 5, 30, 12, 10, 2014);
        Console.WriteLine(agora.ToString());
    }
}
```

Ao executarmos o programa a os valores 10,5,30,12,10 e 2014, são passadas para o construtor da classe *DataHora*, que por sua vez monta a data 12/10/2014 e hora 10:05:30, que é apresentado pelo método *ToString()*

**Exercício 1:**

B1-8 2

A associação pode existir entre :

A)

Classes ou entre objetos.

B)

Apenas entre classes.

C)

Apenas entre objetos.

D)

Apenas entre classes e objetos.

E)

Entre métodos e atributos

Exercício 2:

B1-8 3

O que é agregação ?

A)

Agregações são um tipo especial de herança no qual as duas classes participantes não possuem em nível igual, mas fazem um relacionamento "todo-parte".

B)

Agregações são um tipo especial de associação no qual as duas classes participantes possuem em nível igual, mas fazem um relacionamento "todo-parte".

C)

gregações são um tipo especial de associação no qual as duas classes participantes possuem em nível igual e fazem um relacionamento "todo-parte".

D)

Agregações são um tipo especial de associação no qual as duas classes participantes não possuem em nível igual, mas fazem um relacionamento "todo-parte".

E)

Agregações são um tipo especial de associação no qual os dois objetos participantes não possuem em nível igual, mas fazem um relacionamento "todo-parte".

Exercício 3:

B1-8 4

O que é agregação ?

A)

Agregações são um tipo especial de herança no qual as duas classes participantes não possuem em nível igual, mas fazem um relacionamento "todo-parte".

B)

Agregações são um tipo especial de associação no qual as duas classes participantes possuem em nível igual, mas fazem um relacionamento "todo-parte".

C)

Agregações são um tipo especial de associação no qual as duas classes participantes possuem em nível igual e fazem um relacionamento "todo-parte".

D)

Agregações são um tipo especial de associação no qual as duas classes participantes não possuem em nível igual, mas fazem um relacionamento "todo-parte".

E)

Agregações são um tipo especial de associação no qual os dois objetos participantes não possuem em nível igual, mas fazem um relacionamento "todo-parte".

Exercício 4:

A classe Classe1 abaixo necessita ser totalmente encapsulado, sendo que o atributo total não pode ter acesso externo para modificação.

```
class Classe1
{
    public String nome;
    public float total=0;
    public void incrementa(float valor)
    {
        this.total += valor;
    }
}
class Program
{
    static void Main(string[] args)
    {
        Classe1 c1 = new Classe1();
        c1.nome = "Galileu";
        c1.incrementa(100);
        Console.WriteLine("{0} {1}\n",c1.nome,c1.total);
    }
}
```

Qual das alternativas apresenta o encapsulamento correto?

A)

```
class Classe1
{
    public String nome;
    private float total=0;
    public void incrementa(float valor)
    {
        this.total += valor;
    }
}
class Program
{
    static void Main(string[] args)
    {
        Classe1 c1 = new Classe1();
        c1.nome = "Galileu";
        c1.incrementa(100);
        Console.WriteLine("{0} {1}\n",c1.nome,c1.total);
    }
}
```

B)

```

class Classe1
{
    private String nome;
    private float total=0;
    public void setNome(String nome)
    {
        this.nome = nome;
    }
    public String getNome()
    {
        return this.nome;
    }
    public float getTotal()
    {
        return this.total;
    }
    public void incrementa(float valor)
    {
        this.total += valor;
    }
}

class Program
{
    static void Main(string[] args)
    {
        Classe1 c1 = new Classe1();
        c1.setNome("Galileu");
        c1.incrementa(100);
        Console.WriteLine("{0} {1}\n",c1.getNome(),c1.getTotal());
    }
}

```

C)

```

class Classe1
{
    private String nome;
    private float total=0;
    public void incrementa(float valor)
    {
        this.total += valor;
    }
}

class Program
{
    static void Main(string[] args)
    {
        Classe1 c1 = new Classe1();
        c1.nome = "Galileu";
        c1.incrementa(100);
        Console.WriteLine("{0} {1}\n",c1.nome,c1.total);
    }
}

```

D)


```
class Classe1
{
    private String nome;
    private float total=0;
    public void setNome(String nome)
    {
        this.nome = nome;
    }
    public String getNome()
    {
        return this.nome;
    }
    public float getTotal()
    {
        return this.total;
    }
    public void incrementa(float valor)
    {
        this.total += valor;
    }
}
class Program
{
    static void Main(string[] args)
    {
        Classe1 c1 = new Classe1();
        c1.setNome="Galileu";
        c1.incrementa(100);
        Console.WriteLine("{0} {1}\n",c1.getNome(),c1.getTotal());
    }
}
```

E)

```
class Classe1
{
    private String nome;
    private float total=0;
    private void setNome(String nome)
    {
        this.nome = nome;
    }
    private String getNome()
    {
        return this.nome;
    }
    private float getTotal()
    {
        return this.total;
    }
    private void incrementa(float valor)
    {
        this.total += valor;
    }
}

class Program
{
    static void Main(string[] args)
    {
        Classe1 c1 = new Classe1();
        c1.setNome="Galileu";
        c1.incrementa=100;
        Console.WriteLine("{0} {1}\n",c1.getNome(),c1.getTotal());
    }
}
```

Exercício 5:

B1-8 7

Considere as seguintes classes:

```

class Jogador
{
    public String nome;
    public int numero;
    public Jogador(String nome){
        this.nome = nome;
    }
}
class Time
{
    public Jogador armador, ala1, ala2, pivo1, pivo2;
    public Time(Jogador j1, Jogador j2, Jogador j3, Jogador j4, Jogador j5)
    {
        this.armador = j1;
        this.ala1 = j2;
        this.ala2 = j3;
        this.pivo1 = j4;
        this.pivo2 = j5;
    }
}

```

Qual dos programas abaixo monta o seguinte time:

Posição	Nome	Numero
Armador	Helio Rubens	2
Ala1	Carioquinha	4
Ala2	Marquinhos	7
Pivo1	Adilson	9
Pivo2	Ubiratan	10

A)

```

class Program
{
    static void Main(string[] args)
    {
        Jogador j1 = new Jogador("Helio Rubens",2);
        Jogador j2 = new Jogador("Carioquinha",4);
        Jogador j3 = new Jogador("Marquinhos",7);
        Jogador j4 = new Jogador("Adilson",9);
        Jogador j5 = new Jogador("Ubiratan",10);
        Time selecao = new Time(j1, j2, j3, j4, j5);
    }
}

```

B)

```

class Program
{
    static void Main(string[] args)
    {
        Time selecao = new Time(j1, j2, j3, j4, j5);
        Jogador j1 = new Jogador("Helio Rubens",2);
        Jogador j2 = new Jogador("Carioguinha",4);
        Jogador j3 = new Jogador("Marquinhos",7);
        Jogador j4 = new Jogador("Adilson",9);
        Jogador j5 = new Jogador("Ubiratan",10);
    }
}

```

C)

```

class Program
{
    static void Main(string[] args)
    {
        Time selecao = new Time( Jogador("Helio Rubens"),
                                Jogador("Carioguinha"), Jogador("Marquinhos"), Jogador("Adilson"), Jogador("Ubiratan"));
        selecao.armador.numero = 2;
        selecao.ala1.numero = 4;
        selecao.ala2.numero = 7;
        selecao.pivo1.numero = 9;
        selecao.pivo2.numero = 10;
    }
}

```

D)

```

class Program
{
    static void Main(string[] args)
    {
        Jogador j1 = new Jogador("Helio Rubens");
        Jogador j2 = new Jogador("Carioguinha");
        Jogador j3 = new Jogador("Marquinhos");
        Jogador j4 = new Jogador("Adilson");
        Jogador j5 = new Jogador("Ubiratan");
        Time selecao = new Time(j1, j2, j3, j4, j5);
        selecao.j1.numero = 2;
        selecao.j2.numero = 4;
        selecao.j3.numero = 7;
        selecao.j4.numero = 9;
        selecao.j5.numero = 10;
    }
}

```

E)

```

class Program
{
    static void Main(string[] args)
    {
        Jogador j1 = new Jogador("Helio Rubens");
        Jogador j2 = new Jogador("Carioquinha");
        Jogador j3 = new Jogador("Marquinhos");
        Jogador j4 = new Jogador("Adilson");
        Jogador j5 = new Jogador("Ubiratan");
        Time selecao = new Time(j1, j2, j3, j4, j5);
        selecao.armador.numero = 2;
        selecao.ala1.numero = 4;
        selecao.ala2.numero = 7;
        selecao.pivo1.numero = 9;
        selecao.pivo2.numero = 10;
    }
}

```

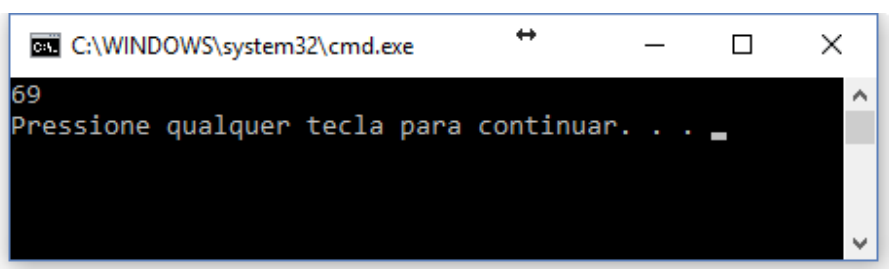
Exercício 6:

B1-8 9

Ao executar o programa abaixo qual a saída, e qual a sequência das linhas executadas?

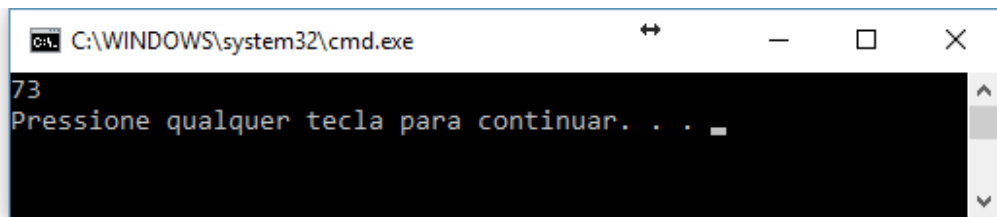
<pre> 1 class Teste 2 { 3 public static int numero = 0; 4 public Teste() 5 : this(3) 6 { 7 numero++; 8 } 9 public Teste(int k) 10 : this(k, 2) 11 { 12 numero += k; 13 } 14 public Teste(int k, int j) 15 { 16 numero *= (k + j); 17 } 18 } </pre>	<pre> 19 class Program 20 { 21 static void Main(string[] args) 22 { 23 Teste p1 = new Teste(1); 24 Teste p2 = new Teste(); 25 Teste p3 = new Teste(); 26 Console.WriteLine(Teste.numero); 27 } 28 } </pre>
---	--

A)



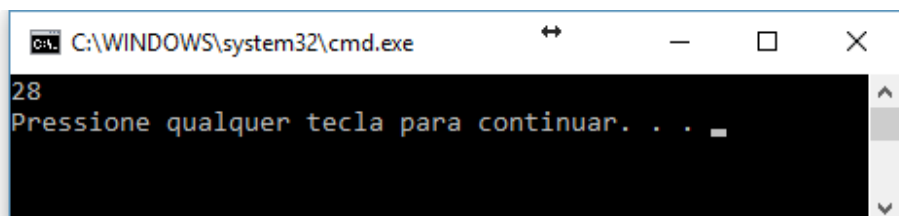
23,p1-4,p1-5,p1-9,p1-10,p1-14,p1-16,p1-12,p1-7,24,p2-9,p2-10,p2-14,p2-16,p2-11,25, p3-4,p3-5,p3-9,p3-10,p3-14,p3-16,p3-12,p3-7,26

B)



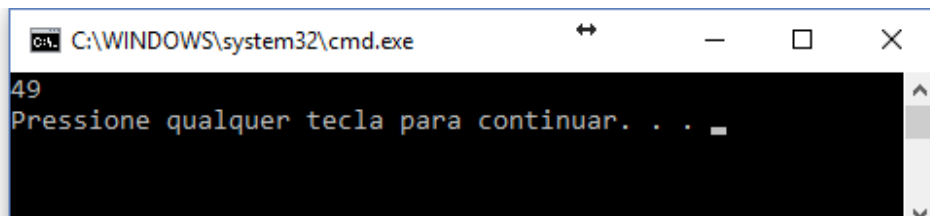
23, p1-4,p1-5,p1-9,p1-10,p1-14,p1-16,p1-12,p1-7,24, p2-4,p2-5,p2-9,p2-10,p2-14,p2-16,p2-12,p2-7,25, p3-9,p3-10,p3-14,p3-16,p3-12,25

C)



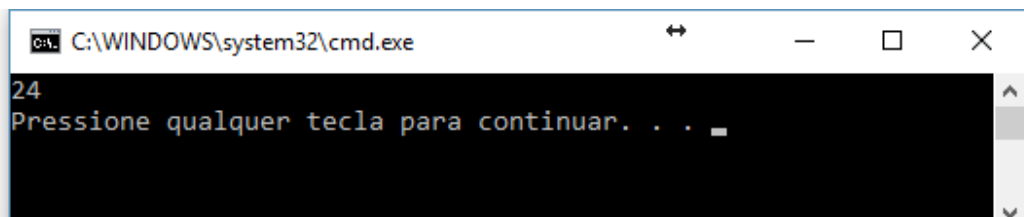
23, p1-9,p1-10,p1-14,p1-16,p1-12,24, p2-4,p2-5,p2-9,p2-10,p2-14,p2-16,p2-12,p2-7, 25, p3-9,p3-10,p3-14,p3-16,p3-12

D)



23, p1-9, p1-10,p1-14,p1-16,p1-12,24,p2-4,p2-5, p2-9, p2-10,p2-14,p2-16, p2-12, p2-7,25, p3-4,p3-5, p3-9, p3-10,p3-14,p3-16, p3-12, p3-7,26

E)



23, p1-9,p1-10,p1-14,p1-16,p1-12,24, p2-9,p2-10,p2-14,p2-16,p2-12, 25, p2-4,p3-5,p3-9,p3-10,p3-14,p3-16,p3-12,p3-7, 26